Operating System Note

Made By Masud Rana Mushfiq

Contents at a Glance:
This note covers the following major topics:

(OS Lecture 01-06)
Slide 01 — Basics: POST, BIOS, MBR, OS Definition
Slide 02 — Common Operating Systems & OS Functions
Slide 03 — Types of Operating Systems
Slide 04 — Services of an Operating System
Slide 05 — Processes, PCB, Scheduling, Context Switching, CPU Scheduling
Slide 06 — Process Scheduling

(Slide of Book Start From Chapter 05) Page: 17
Chapter 5 — CPU Scheduling (Burst Cycle, Scheduler, Preemption, Dispatcher)
Chapter 6 — Synchronization Page: 29
Chapter 8 — Deadlocks (Prevention, Avoidance, Detection, Recovery) Page: 33
Chapter 9 — Main Memory (Protection, Paging, TLB, Swapping) Page: 41
Chapter 10 — Virtual Memory (Introduction) Page: 55
Chapter 13 — File-System Interface Page: 60

Book: *Operating System Concepts – 10th Edition by Silberschatz*

Slide 01:

POST: It is a process that runs when a computer is turned on to check if the hardware is working properly.
Bios Chip: The BIOS (Basic Input/Output System) chip is a firmware stored on the motherboard that starts the computer.
Master Boot Record (MBR)
The Master Boot Record (MBR) is a small section of a storage device that contains boot information and partition details. It is used by the system to locate and load the operating system during startup.

Operating System:
An Operating System (OS) is software that manages a computer's hardware and software resources.

- It is an intermediary between computer hardware and user.
- It controls all activities, shares resources among applications, and provides basic functions like processor, memory, and file management.
- It is a low level software that running all time and acts like a government for the system, managing different resources through various "departments."

Slide 02:

Some Common Operating Systems:

Windows OS
Developer: Microsoft
Key Features: User-friendly interface, software compatibility, hardware support, Strong gaming support.
Advantages: Easy to use for most users, Broad support from third-party applications, Frequent updates and support.
Typical Use Cases: Personal computing, Business environment, Gaming.
Examples: DOS, Windows 3.1, Windows XP, Windows 11

macOS (mac for Macintosh)
Developer: Apple.
Key Features: Smooth, intuitive user interface, Strong integration with other Apple products, Robust security features, High performance and stability.
Advantages: Optimized for Apple hardware, Seamless experience across Apple ecosystem, Superior graphics and multimedia capabilities.
Typical Use Cases: Creative industries (design, video editing, music production), Personal computing, Professional environments.
Examples: system 1, system 2,.....macOS 8, macOS 9

Unix

Developer: Originally AT&T Bell Labs, various commercial and open-source versions available

Key Features: Multiuser and multitasking capabilities, Strong security and stability, Powerful command-line interface, Portability across different hardware platforms

Advantages: Reliable and robust performance, Suitable for high-performance computing and servers, Extensive support for networking

Typical Use Cases: Servers and workstations, Development environments, Research and academic settings

Examples: Unix 1, Unix 2, Solaris, IRIS.

Linux

Developer: No specific developer, it is Community-driven (various distributions). First developed by Linus Torvadls.

Key Features: Open-source and highly customizable, Robust security and stability, Lightweight and can run on older hardware (Ubuntu, Fedora, Debian).

Advantages: Free to use and distribute, Strong community support, Suitable for servers and development environments.

Typical Use Cases: Servers and data centers, Development and programming, Personal computing for tech enthusiasts.

Linux was created in 1991 by Linus Torvalds as a free UNIX-like kernel and, combined with GNU tools, became a complete open-source operating system.

Functions of an Operating System (OS)

1. **Resource Management:**
Manages hardware and software resources when multiple users access the system, reducing system load.
2. **Process Management:**
Handles tasks like scheduling and terminating processes using CPU scheduling algorithms.
3. **Storage Management:**
Manages data storage through file systems (NFS, CIFS, CFS) and organizes data on hard disks.
4. **Memory Management:**
Tracks memory usage, allocates and deallocates memory to processes as needed.
5. **Security/Privacy Management:**
Protects data using methods like passwords and authentication (e.g., Kerberos in Windows).
6. **Device Management:** Controls input/output devices, deciding which process can use a device, when, and for how long.

Fundamental Goals of an Operating System (OS)

1. Efficient Use: Ensure that computer resources like CPU, memory, and I/O devices are used efficiently. The OS manages resources to avoid waste and maximize performance.

2. User Convenience: Make the computer easy to use. Modern OS provide graphical interfaces with icons and mouse interactions, replacing complex command-line operations.

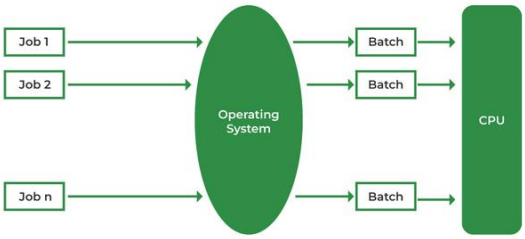
3. Non-Interference: Prevent programs or users from interfering with each other. The OS allocates resources safely and blocks unauthorized access.

Slide 03:

Some Type Of Operating System:

Batch Operating System

A Batch Operating System does not interact directly with the computer. An operator collects similar jobs, groups them into batches, and the system executes them together efficiently.



Advantages:
Multiple users can share the system
Minimal idle time
Easy to manage large repetitive work

Disadvantages:
CPU may remain idle during I/O operations
Jobs must wait if any job fails
Average response time is high as jobs are processed one by one

Multiprogramming Operating System

A **Multiprogramming Operating System** is one where **multiple programs are kept in main memory at the same time**, and the CPU executes **one program while others wait.**
When one program is idle (e.g., waiting for I/O), the CPU switches to another program.

Key Idea:
Many programs in memory, CPU is **never idle,** Improves resource utilization.

Advantages

1. **Better CPU Utilization:** CPU always stays busy
2. **Improved System Performance:** More work is done in less time
3. **Reduced Response Time:** Faster execution compared to single-program systems

Multitasking / Time-Sharing Operating System

A **Time-Sharing Operating System** is a type of **multiprogramming system** where multiple tasks are executed by the CPU **one by one in a very fast cycle** **(round-robin manner)**.

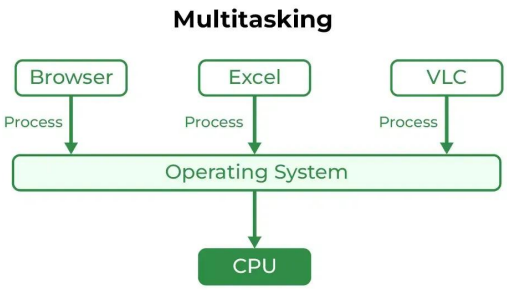
Each task gets a small fixed time called a **time quantum**. After that, the CPU switches to the next task.

Key Idea

- Many users/tasks share one system
- CPU time is divided into small parts
- Fast switching creates the illusion of simultaneous execution

Features

- Uses **Round Robin scheduling**
- Supports **multiple users or tasks**
- Provides **interactive system**



Advantages

1. **Equal Opportunity:** Each task gets CPU time
2. **Better CPU Utilization:** Reduces idle time
3. **Resource Sharing:** CPU, memory, devices shared efficiently
4. **Improved Productivity:** Multiple users work at the same time
5. **Better User Experience:** Fast response, interactive use

Disadvantages

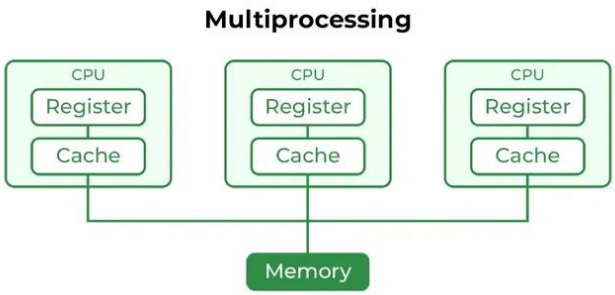
1. **Complex System:** Difficult to design and manage
2. **High Overhead:** Frequent context switching
3. **Security Risks:** Multiple users increase risk
4. **Reliability Issues:** System failure affects all users
5. **Data Communication Issues:** Possible delays and conflicts

Multiprocessing Operating System

A **Multiprocessing Operating System** is one that uses **more than one CPU** **(processor)** to execute processes **simultaneously**.

Key Idea

- Multiple CPUs
- Parallel execution
- Faster processing



Advantages

1. **Increased Throughput:** More tasks completed in less time
2. **Parallel Processing:** Multiple processes run at the same time
3. **Reliability:** If one CPU fails, others can continue
4. **Better Performance:** Faster execution of programs

Disadvantages

1. **Complex Design:** Difficult to manage multiple CPUs
2. **Costly:** Requires more hardware
3. **Synchronization Issues:** Managing shared data is difficult

Distributed Operating System

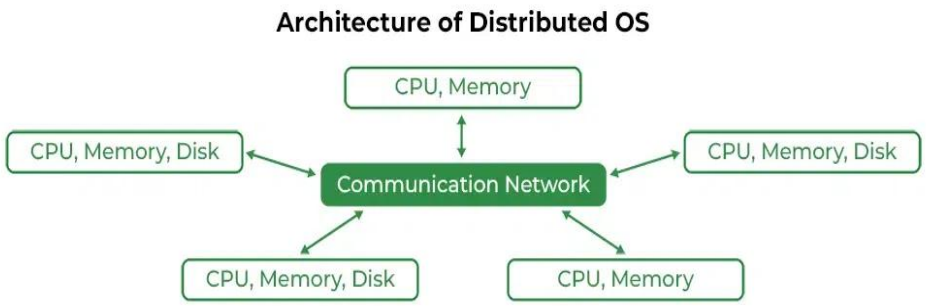
A **Distributed Operating System** connects multiple independent computers through a network, allowing users to access files or software on other systems.

Advantages:

- Failure of one system does not affect others
- Shared resources speed up computation
- Easily scalable

Disadvantages:

- Main network failure stops communication
- Expensive to set up
- Software is complex



Real-Time Operating System (RTOS)

A **Real-Time Operating System (RTOS)** is designed to process and respond to inputs within a very short, predictable **response time**. It is used in systems with strict timing requirements, such as missile systems, air traffic control, and robots.

Types of RTOS:

Hard Real-Time OS:

Timing is critical; missing a deadline can cause serious consequences (e.g., medical devices, airbags, defense systems). It is complex design and high cost.

Soft Real-Time OS:

Timing is important but missing a deadline does not cause system failure (e.g., multimedia, online games). Easy to implement and cost effective.

Advantages of Hard RTOS:

1. Guaranteed response time
2. Suitable for critical applications

Advantages of Soft RTOS:

1. Flexible timing
2. Useful where strict deadlines are not critical

Mobile Operating System

A **Mobile Operating System** is an OS designed for **mobile devices** like smartphones and tablets. It manages hardware, runs apps, and provides a user-friendly interface.

Key Features

1. **User Interface (UI):** Touch-based (tap, swipe, pinch)
2. **Multitasking:** Run multiple apps and switch easily
3. **Connectivity:** Supports Wi-Fi, Bluetooth, cellular, NFC
4. **Application Management:** App store for installing and updating apps
5. **Resource Management:** Efficient use of CPU, RAM, and battery

Types of Mobile OS

1. Android
    1. Developed by Google
    2. Open-source, widely used
2. iOS
    1. Developed by Apple Inc.
    2. Secure, smooth, limited customization
3. HarmonyOS
    1. Developed by Huawei
    2. Works across multiple devices
4. KaiOS
    1. Used in basic phones (supports apps like WhatsApp)
5. Tizen
    1. Developed by Samsung
    2. Used in Samsung devices

Embedded Operating Systems

Embedded systems are specialized computers designed for specific tasks within larger devices. Unlike general-purpose computers, they focus on real-time performance, low power use, compact size, and reliability. Embedded OS manages their hardware and software efficiently.

Key Characteristics of Embedded Systems

Dedicated Functionality: Performs specific tasks efficiently and reliably.
Real-Time Operation: Processes inputs and outputs within strict time limits.
Resource Constraints: Optimized for limited memory, CPU, and energy.
Integration and Miniaturization: Compact and built into larger systems.
Reliability and Robustness: Works reliably in critical applications.
Specialized Hardware and Software: Uses custom microcontrollers, RTOS, or ASIC.

Advantages of Embedded Systems

High Efficiency and Performance: Embedded systems are highly optimized and handle time-critical tasks quickly.
Low Power Consumption: They use energy efficiently, making them ideal for battery-powered devices with longer life.
Compact Size: Embedded systems are small and save space, suitable for multi-functional devices.
Cost-Effectiveness: They require fewer components and can be mass-produced to reduce costs.
Reliability and Stability: Dedicated functionality makes them reliable and able to run continuously with minimal maintenance.

Slide 04:

Services of Operating System

- ✓ Program execution
- ✓ Input Output Operations
- ✓ Communication between Process
- ✓ File Management
- ✓ Memory Management
- ✓ Process Management
- ✓ Time Management
- ✓ Resource Management
- ✓ Security and Privacy
- ✓ User Interface
- ✓ Networking
- ✓ Error handling

Their Explain:

Program Execution

The OS manages program execution by loading programs into memory and controlling their order using CPU scheduling algorithms like FCFS and SJF. It ensures smooth execution and prevents deadlocks.

Input-Output Operations

The OS manages communication between programs and devices through device drivers and provides controlled access to input-output devices.

Communication Between Processes

The OS handles data transfer between processes, even if they are on different computers connected via a network.

Memory Management

The OS allocates memory to programs based on their requirements, prevents unnecessary usage, and ensures efficient memory utilization.

File Management

The OS manages files, grants access permissions (read, write, etc.), and decides how data is stored and manipulated on storage devices.

Process Management

The OS schedules programs (like a chef managing a kitchen) to ensure all processes get CPU time and run efficiently without delays.

Security and Privacy

Security: Protects the system from unauthorized access, viruses, and hackers using firewalls and antivirus software.
Privacy: Keeps user information safe, like locking essential data from others.

Resource Management

The OS manages system resources like CPU, memory, and I/O devices, ensuring fair allocation among processes.

User Interface

The OS provides a way for users to interact, either through a **command-line interface (CLI)** or a **graphical user interface (GUI)** with windows, menus, and mouse control.

Networking

The OS enables communication between devices, manages internet connections, and handles data transmission.

Error Handling
The OS detects, prevents, and fixes errors in CPU, I/O devices, and processes, avoiding deadlocks and system crashes.

Time Management
The OS schedules program execution like a traffic light, controlling when processes wait, are ready, or execute to ensure smooth operation.

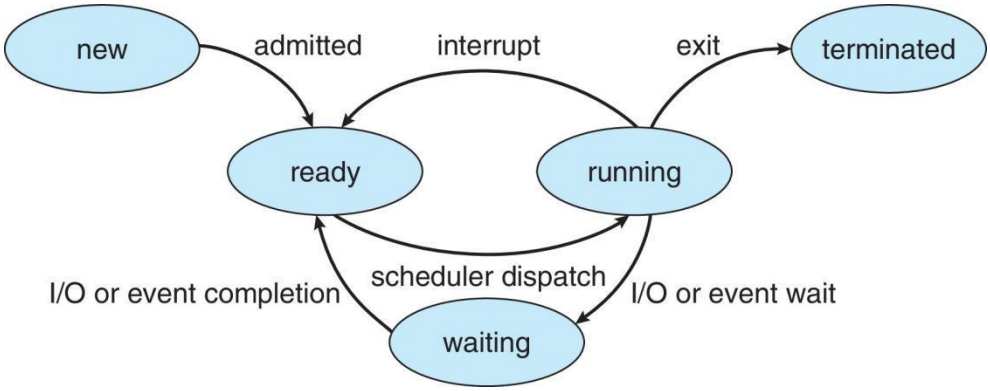
Slide 05:

Process:
A process is a program in execution. Its instructions run sequentially, and in a single-processor system, a process cannot execute multiple instructions in parallel.

Multiple parts of a Process
1. The program code, also called text section
2. Current activity including program counter, processor registers
3. Stack containing temporary data
4. Function parameters, return addresses, local variables
5. Data section containing global variables
6. Heap containing memory dynamically allocated during run time

- A **program** is a passive entity stored on disk as an executable file, while a **process** is an active entity in execution.
- A program becomes a **process** when it is loaded into memory.
- Program execution can start through GUI actions (mouse clicks) or command-line commands.
- A single program can create **multiple processes**.
- Multiple users or a single user can run several instances of the same program at the same time.

As a process executes, it changes state:
New: The process is being created
Ready: The process is waiting to be assigned to a processor
Running: Instructions are being executed
Waiting: The process is waiting for some event to occur
Terminated: The process has finished execution



A thread is the smallest unit of execution within a process.
A process can have multiple threads, each performing a part of the task.

Process Control Block (PCB) also called Task Control Block:
A **Process Control Block (PCB)** is a data structure used by the operating system to store all information related to a process.

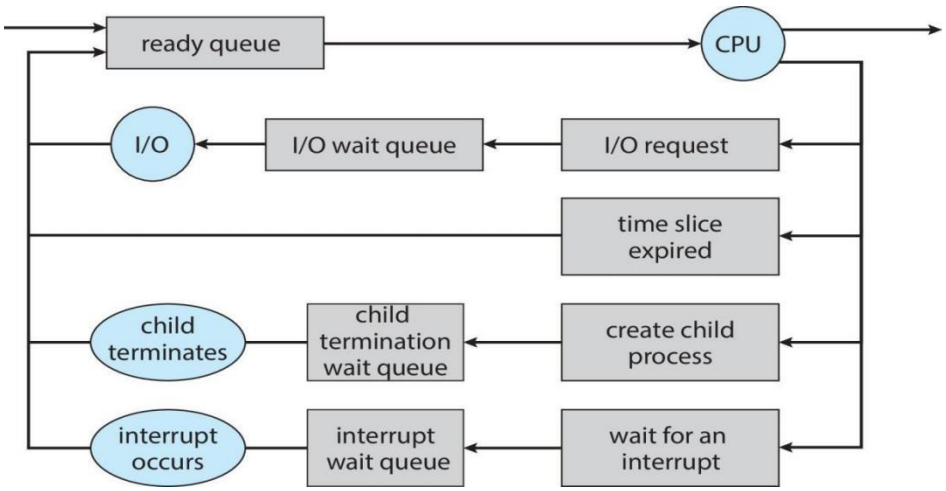
It includes:
1. **Process ID:** Unique identifier of the process
2. **Process State:** Current state (running, waiting, etc.)
3. **Program Counter:** Address of the next instruction to execute
4. **CPU Registers:** Contents of all process-related registers
5. **CPU Scheduling Information:** Priority and scheduling queue pointers
6. **Memory Management Information:** Memory allocated to the process
7. **Accounting Information:** CPU time used and execution time limits
8. **I/O Status Information:** Allocated I/O devices and list of open files

Process Scheduler
The **process scheduler** selects one process from the available processes for execution on the CPU core. Its main goal is to maximize CPU utilization and quickly switch processes onto the CPU.

The scheduler maintains **scheduling queues**:
- **Ready Queue:** Contains processes in main memory that are ready to execute.
- **Wait Queue:** Contains processes waiting for an event such as I/O.

Processes move (migrate) between these queues during their execution

Representation of Process Scheduling: The Queueing Diagram



The queueing diagram shows how a process is placed in different types of queues during its life cycle.

Context Switching

Context switching is the process of saving the state of the currently running process and loading the state of another process so it can execute.
Context switch time is **pure overhead**, as no useful work is done during switching.
A more complex operating system and larger **PCB** increase the context-switch time.

Context Switching Triggers

Context switching occurs due to the following three triggers:

1. Interrupts

When an interrupt occurs, the CPU switches context to handle it quickly and then resumes the previous process.
Example: While watching a video, plugging in a USB device causes the CPU to temporarily handle the USB detection and then return to the video.

2. Multitasking

In multitasking, the OS switches the CPU from one process to another so multiple programs can run smoothly.
Example: While editing a document and listening to music, starting a heavy application causes the CPU to switch between tasks efficiently.

3. User/Kernel Switch
Context switching occurs when the CPU switches between user mode and kernel mode for privileged operations.
Example: When a user program requests to read a file, the CPU switches to kernel mode to perform the operation safely, then returns to user mode.

CPU Scheduling

CPU scheduling is important in multi-programming operating systems because multiple processes may be ready to run at the same time. One of the main responsibilities of the operating system is to decide which process will execute on the CPU.
Process schedulers are OS components that determine the order of process execution and manage how CPU time is shared among competing processes. Since the CPU can execute only one process at a time, scheduling ensures fair and efficient CPU usage.

Objectives of CPU Scheduling:
- Maximize CPU utilization
- Minimize process response time and waiting time

Interprocess Communication (IPC)

What is IPC?
- Processes are like **separate workers** in a computer; each does its own job.
- They can be:
  1. **Independent:** work alone.
  2. **Cooperating:** can affect or be affected by other processes, often by **sharing data**.
- **IPC** is the mechanism for cooperating processes to **communicate safely**.
- Without IPC, processes are isolated; with IPC, they can:
  1. **Share data**
  2. **Work together faster**
  3. **Break a big task into smaller parts**

Why Cooperate?
Cooperation is useful for:
1. **Information sharing**
2. **Speedup of computations**
3. **Modularity**
4. **Convenience**

Models of IPC

1. **Shared Memory**: processes communicate by **reading/writing a common memory area**; synchronization is the responsibility of the processes.
2. **Message Passing**: processes communicate by **sending and receiving messages** via the OS.

Producer-Consumer Problem

- **A classic IPC example**:
  - **Producer:** creates data (tasks, files, or items).
  - **Consumer:** uses or consumes the data.
  - **Buffer:** temporary storage area for items.
- **Rules for safe sharing:**
  - Producer cannot add items if the buffer is full.
  - Consumer cannot take items if the buffer is empty.
  - Both must coordinate to avoid mistakes.
- **Variations:**
  1. **Unbounded buffer:** unlimited storage; producer never waits, consumer waits only if empty.
  2. **Bounded buffer:** fixed storage; producer waits if full, consumer waits if empty.

Shared Memory Solution (Bounded Buffer)

- **Buffer setup:**
  - Array of fixed size.
  - in index for producer, out index for consumer.
- **Producer loop:**
  - Wait if buffer is full.
  - Add item to buffer and update in.
- **Consumer loop:**
  - Wait if buffer is empty.
  - Remove item from buffer and update out.
- **Counter-based full-buffer solution:**
  - Use a **counter** to track the number of filled slots.
  - Producer increments the counter after producing.
  - Consumer decrements the counter after consuming.

Race Condition

- Occurs when **multiple processes access shared data simultaneously** without proper synchronization.
- Example: producer and consumer both updating a **counter** at the same time can lead to **lost updates**.
- **Solution:** use **synchronization mechanisms** (mutexes, semaphores) to avoid conflicts.

Critical Region

- A **critical region** is a **part of a program where shared resources (like shared memory) are accessed**.
- Only **one process should enter at a time** to avoid conflicts.
- If multiple processes enter simultaneously, it can cause **race conditions**.

Mutual Exclusion with Busy Waiting

Mutual exclusion ensures that only **one process can enter the critical region at a time**.

Busy waiting is when a process **keeps checking** if it can enter the critical region instead of waiting passively.

Methods:

1. **Disabling Interrupts**
   - Temporarily **stop all other processes** from interrupting.
   - Ensures the current process can access shared memory safely.
   - **Problem:** Not suitable for multiprocessor systems because other CPUs may still access memory.
2. **Lock Variables**
   - Use a shared variable (like a **flag**) to indicate if a process is in the critical region.
   - Example:
     - lock = 0 → no one in critical region
     - lock = 1 → someone is inside
   - Process **waits (busy wait)** until lock becomes 0.
   - **Problem:** Can lead to race conditions if not implemented carefully.
3. **Strict Alternation**
   - Processes **take turns** to enter the critical region.
   - Works like **round-robin**: Process A goes, then Process B, and so on.
   - **Problem:** Can reduce concurrency because a process may wait even if the other process doesn't need the critical region.

Slide 06:

Process Scheduling

Process scheduling is the activity of the operating system that removes the currently running process from the CPU and selects another process based on a scheduling strategy. During its lifetime, a process moves among different scheduling queues such as the **ready queue**, **waiting queue**, and **device queue**.

Types of Process Schedulers

There are three types of process schedulers in an operating system:

1. Long-Term Scheduler (Job Scheduler)
Loads processes from disk to main memory, moving them from **Job Queue → Ready Queue.**
Controls the degree of multi-programming (number of processes in memory).
Carefully selects a mix of I/O-bound and CPU-bound processes.
Runs less frequently than other schedulers.

2. Short-Term Scheduler (CPU Scheduler)
Selects a process from the r**eady queue to run on the CPU.**
Uses scheduling algorithms to allocate CPU efficiently and avoid starvation.
Calls the dispatcher to switch processes.
Fastest scheduler since it runs frequently.
Main goal: maximize CPU utilization.

3. Medium-Term Scheduler
Moves processes from memory to disk (suspension) and back to reduce memory load.
Reduces the degree of multiprogramming when processes are waiting (e.g., for I/O).
Brings suspended processes back into memory when needed to resume execution.
Speed: faster than long-term, slower than short-term scheduler.

Chapter 5: CPU Scheduling

1. Why CPU Utilization and Multiprogramming

CPU Utilization = percentage of time CPU is busy
Goal: maximize CPU utilization

Multiprogramming allows multiple processes to **reside in memory**.
When one process is waiting for I/O, another process uses the CPU.

Result: CPU remains busy most of the time. System efficiency increases

2. CPU and I/O Burst Cycle

Process execution is not continuous. It follows a repeating cycle:
CPU Burst → I/O Burst → CPU Burst → I/O Burst → ... → Termination

Definitions:
CPU Burst: Time during which process **executes instructions**
I/O Burst: Time during which process waits for I/O operation

Key Point: Every process alternates between computation and waiting.

3. Importance of CPU Burst

CPU scheduling decisions depend mainly on Length of CPU bursts as CPU burst length helps the OS decide which process to run first for better performance.

- ✓ Shorter bursts → faster completion
- ✓ Longer bursts → more CPU occupation

4. CPU Burst Time Distribution: Observation from real systems is:

1. Large number of short CPU bursts, Most processes are short (I/O-bound)
2. Small number of long CPU bursts, Few processes are long (CPU-bound)

5. Types of Processes

(a) An **Input-Output bound** process spends most of its time waiting for input/output operations rather than using the CPU.

1. Many short CPU bursts
2. Frequent I/O operations

Examples: text editor, web browser

(b) A **CPU-bound process** spends most of its time performing computations and keeps the CPU busy.

1. Few but long CPU bursts
2. Less I/O activity

Examples: scientific programs, data processing

6. Significance of Scheduling

This distribution helps in designing efficient scheduling algorithms:

1. Algorithms like SJF (Shortest Job First) perform well
2. Short processes are executed quickly

Executing short CPU bursts first allows processes to finish earlier, which reduces both waiting time and turnaround time for the system.

CPU scheduling is designed based on the fact that most processes have short CPU bursts, ensuring efficient and responsive system performance.

CPU Scheduler, Preemption & Dispatcher

CPU Scheduler

The CPU scheduler selects one process from the ready queue and allocates the CPU to it. Since multiple processes may be ready, the scheduler decides the execution order based on a scheduling policy.

When Scheduling Occurs

Scheduling decisions happen **when a process changes state:**

- Running → Waiting (e.g., I/O request)
- Running → Ready (preemption)
- Waiting → Ready (I/O completion)
- Process Termination

In the first and last cases, the system must choose a new process. In the other cases, it has a choice.

Preemptive vs Non-Preemptive Scheduling

Non-preemptive scheduling

1. Process use CPU until it finishes or waits for I/O
2. No forced interruption to stop use CPU by the process

Preemptive scheduling

1. OS can interrupt a running process to stop use CPU
2. Improves responsiveness
3. Used in modern systems like Windows and Linux

Race Condition: A **race condition** occurs when multiple processes **share and access the same data at the same time** without proper synchronization.

What happens conceptually:

- One process starts **updating shared data**.
- Before it finishes, it gets **interrupted**.
- Another process **reads or modifies the same data**.
- This leads to **incorrect or inconsistent results**.

Dispatcher: The dispatcher is the part of the OS that gives the CPU to the selected process after scheduling.

What it does

- **Context Switching:** saves the current process state and loads the next process state.
- **Switch to User Mode:** changes from kernel mode to user mode before execution.
- **Resume Process:** starts or continues execution of the selected process.

Dispatch Latency

Time required to switch CPU from one process to another. It should be very small for better performance

CPU Scheduling Criteria

The performance of a CPU scheduling algorithm is measured using the following criteria:

1. **CPU Utilization** – Keep the CPU as busy as possible to maximize efficiency.
2. **Throughput** – Number of processes completed per unit time.
3. **Turnaround Time** – Total time taken to execute a process, from submission to completion.
4. **Waiting Time** – Time a process spends waiting in the ready queue.
5. **Response Time** – Time from when a request is submitted to when the first response is produced.

Optimization Goals

An ideal scheduling algorithm should aim to:

1) Maximize CPU utilization
2) Maximize throughput
3) Minimize turnaround time
4) Minimize waiting time
5) Minimize response time

CPU Scheduling Algorithm: FCFS and SJF

First-Come, First-Served (FCFS) Scheduling

Processes are executed in the order they arrive.

Example 1: P1 = 24, P2 = 3, P3 = 3, Arrival Order: P1, P2, P3
Waiting times: P1 = 0, P2 = 24, P3 = 27
Average waiting time = (0 + 24 + 27)/3 = 17

Example 2: Arrival Order: P2, P3, P1
Waiting times: P1 = 6, P2 = 0, P3 = 3
Average waiting time = (6 + 0 + 3)/3 = 3

Key Point: FCFS can cause the **convoy effect**(Short processes get "stuck behind" a long process) when short processes wait behind long ones.

Shortest-Job-First (SJF) Scheduling

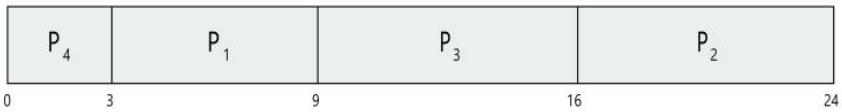
Schedule the process with the shortest next CPU burst first, longest are then.
Minimizes average waiting time (optimal for a given set of processes).

Preemptive version: Shortest-Remaining-Time-First (SRTF).
Next CPU burst length is usually estimated using previous burst history or user input.

Example of SJF
Processes: P1 = 6, P2 = 8, P3 = 7, P4 = 3
Average waiting time = (3 + 16 + 9 + 0)/4 = 7

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

- SJF scheduling chart



- Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

In **Shortest-Job-First (SJF)** scheduling, the arrival order of processes stays the same, but the CPU always selects the process with the shortest next CPU burst first.

This means:

- Short processes get the CPU earlier, even if they arrived later than a long process.
- Long processes wait, but because short processes finish quickly, the average waiting time decreases.
- This is why SJF is considered optimal for minimizing average waiting time compared to FCFS.

In simple terms: short jobs "jump ahead" in the queue, reducing the total waiting time for all processes.

Estimating Next CPU Burst

The OS tries to predict the next CPU burst time of a process to make better scheduling decisions (like in SJF). It uses exponential averaging to combine past and recent CPU burst times.

$$T_{n+1} = \alpha \times t_n + (1 - \alpha) \times T_n,$$ here α usually set to 0.5

t_n = actual last CPU burst
T_n = previous estimate
t_{n+1} = new predicted CPU burst

The prediction is a mix of past estimate and recent actual value.

Special cases:

- $\alpha = 0 \rightarrow$ only past prediction counts
- $\alpha = 1 \rightarrow$ only last CPU burst counts

The formula gives a weighted sum where older bursts have progressively less influence.

Shortest Remaining Time First (SRTF) Scheduling

- SRTF is the preemptive version of Shortest Job Next (SJN/SJF).
- Whenever a new process arrives in the ready queue, the scheduler reevaluates **which process has the shortest remaining CPU burst** and may temporarily stop the running process to let a shorter job run, which reduces the overall average waiting time.
- SRTF usually gives lower average waiting time than non-preemptive SJF for the same set of processes because it always favors the shortest remaining burst.

Round Robin (RR) Scheduling

In Round Robin, each process is assigned a **time quantum (q)**, typically 10–100 milliseconds. A process runs for at most q units, then is preempted and placed at the end of the ready queue.

With n processes, each gets roughly **1/n of CPU time** in turns, and no process waits more than **(n-1) × q** time units.
Timer interrupts control the switching between processes.

Performance Considerations

1. Large q → behaves like FCFS (longer response for short jobs).
2. Small q → frequent context switches, higher overhead, but better responsiveness.
3. q must be much larger than context switch time to avoid excessive overhead.
4. Typically, q = 10–100 ms, context switch < 10 μs.

Example

Processes: P1 = 24, P2 = 3, P3 = 3, Time Quantum = 4
Round Robin ensures each process gets CPU in turns
Average turnaround time is usually higher than SJF, but response time improves.

Key Point

About 80% of CPU bursts should be shorter than q for efficient RR scheduling.

Priority Scheduling

1) Each process is assigned a priority number (smaller number → higher priority).
2) CPU is allocated to the highest-priority process.
3) Can be preemptive or non-preemptive.
4) SJF is a special case of priority scheduling where priority is the inverse of the predicted next CPU burst.

But Here Problem May Arise like :

Starvation: Low-priority processes may never get CPU.
Solution – Aging: Gradually increase the priority of waiting processes to prevent starvation.

Priority Scheduling with Round-Robin

Normally Run the process with the highest priority
Among processes with the same priority, use Round Robin.

Multilevel Queue & Multilevel Feedback Queue Scheduling

Multilevel Queue (MLQ)

1. The ready queue is divided into multiple separate queues.
2. Each queue can have its own scheduling algorithm.
3. Processes are assigned to queues based on priority, type, or service needs.
4. CPU always serves the highest-priority non-empty queue first.

Multilevel Feedback Queue (MLFQ)

Processes can move between queues based on behavior. You can say scheduling decision for a process depends on its previous behavior.

MLFQ Scheduler parameters:

1) Number of queues
2) Scheduling algorithm for each queue
3) Rules for promotion and demotion of processes
4) Queue selection when a process needs CPU
5) Aging can be implemented to prevent starvation.

Example of MLFQ

Queues:

- Q0 – Round Robin (time quantum 8 ms)
- Q1 – Round Robin (time quantum 16 ms)
- Q2 – FCFS

Scheduling:

- New process enters Q0 → served in RR for 8 ms
- If unfinished → moved to Q1 → served in RR for 16 ms
- If still unfinished → moved to Q2 → served FCFS

Thread Scheduling

When an operating system supports threads, the CPU scheduler schedules individual threads rather than whole processes. There is a distinction between **user-level** threads and **kernel-level** threads.

User Threads and Kernel Threads

Threads are the **basic units of execution inside a process**, and this topic explains **who manages them (user or kernel) and how they are mapped** to achieve concurrency and parallelism.

User Threads and Kernel Threads

User Threads

- Managed by a **user-level thread library**, not by the kernel.
- Fast and lightweight, but **kernel is unaware**, so blocking affects all threads.
- **Thread libraries:**
  - POSIX Pthreads
  - Windows threads
  - Java threads

Kernel Threads

- Managed directly by the **operating system kernel.**
- Supports **true parallelism and better scheduling** by the OS.
- **Examples:** Windows, Linux, Mac OS X, iOS, Android

Multithreading Models

These models define **how user threads are mapped to kernel threads**, affecting performance and concurrency.

Many-to-One Model

- Many user threads mapped to **one kernel thread**.
- Simple but no parallelism; **one blocking stops all threads**.
- **Examples:**
  - Solaris Green Threads
  - GNU Portable Threads

One-to-One Model

- Each user thread maps to **one kernel thread**.
- Allows **high concurrency**, but creates overhead due to many kernel threads.
- **Examples:** Windows, Linux

Many-to-Many Model

- Many user threads mapped to **multiple kernel threads**.
- Provides **flexibility and efficient resource use**.
- Example: Windows (ThreadFiber package)

Two-Level Model

- Similar to Many-to-Many, but allows **binding specific user threads to kernel threads**.
- Combines **flexibility with control**.

Multiple-Processor Scheduling

Scheduling becomes more complex when multiple CPUs are available.

Types of Multiprocessing

Multiprocessing means using **more than one CPU** to execute processes simultaneously.

1. **Asymmetric Multiprocessing** (AMP)
   - **One CPU (master)** controls scheduling and system management.
   - Other CPUs (slaves) **execute tasks assigned by the master**.
   - **Simple but less flexible** for modern systems.
2. **Symmetric Multiprocessing** (SMP)
   - **All CPUs are equal**, no permanent master.
   - Each CPU can **pick processes to run**, allowing better parallelism.
   - **SMP Strategies:**
     - **Global shared run queue:** all CPUs pick processes from one common queue.
     - **Per-CPU run queue:** each CPU has its own queue to reduce contention.

Multiprocessor systems can include:

1. Multicore CPUs
2. Multithreaded cores
3. NUMA (Non-Uniform Memory Access) systems

Multicore Processors

Multiple cores on the same chip → faster, lower power consumption.
Multiple hardware threads per core allow progress while one thread is stalled for memory.

Chip Multithreading (CMT) / Intel Hyper-Threading:
Example: 4-core CPU × 2 threads per core → OS sees 8 logical processors

Two Levels of Scheduling
OS Level: Decides which software thread runs on which logical CPU.
Core Level: Each core decides which hardware thread to run on the physical core.

Load Balancing

Distributing the workload evenly across all available CPUs so that no CPU is idle while others are overloaded.

1. **Push Migration:** Overloaded CPU pushes tasks to others CPU.
2. **Pull Migration:** Idle CPU pulls tasks from busy CPU.

Processor Affinity: Processor affinity is the tendency for a thread to run on the same CPU it ran on previously. A thread tends to run on the same CPU where its cache contains relevant data.

1) Soft Affinity: OS tries to keep thread on same CPU (no guarantee).
2) Hard Affinity: Process specifies which CPUs it can run on.

Load balancing may move threads, but cache contents may be lost, reducing efficiency.

Real-Time CPU Scheduling

Real-time scheduling ensures tasks meet timing constraints; some tasks must meet deadlines (hard), others have priority but no strict guarantee (soft).

Types of Real-Time Systems:

Soft Real-Time: Critical tasks have high priority, but deadlines are not guaranteed.
Hard Real-Time: Tasks must meet deadlines; missing a deadline can cause system failure.

Event Latency: Time from when an event occurs to when it is serviced.
Interrupt Latency: Time from interrupt arrival to start of its service routine.
Dispatch Latency: Time to preempt the current process and switch CPU to another process.

Conflict Phase of Dispatch Latency

Delay in switching to a high-priority process due to system conflicts.

- **Kernel Mode Issue:**
  Process in kernel mode cannot be preempted immediately → delay
- **Priority Inversion:**
  Low-priority process holds resource needed by high-priority → blocking

Solution: Use priority inheritance to reduce delay

Priority-Based Scheduling

CPU is assigned based on priority (higher priority → executes first)

1) Must be preemptive for real-time systems
2) Guarantees only soft real-time
3) Hard real-time: must meet strict deadlines

Rate Monotonic Scheduling (RMS): Priority based on period (p)

1. Shorter period → higher priority
2. Longer period → lower priority

Missed Deadlines in RMS: A process may fail to complete before its deadline

1. Happens under high CPU load
2. Example: lower priority task gets delayed

Earliest Deadline First (EDF): Priority based on deadline (d)

1) Earlier deadline → higher priority
2) Later deadline → lower priority

Different Operating System Scheduling

Linux Scheduling: Before Version 2.5

Concept: Priority-based preemptive scheduling with O(1) time

1. Two priorities: Real-time (0–99), Normal (100–140)
2. Lower number → higher priority
3. Uses active & expired queues
4. Issue: poor response for interactive processes

After Version 2.6.23 (CFS)
Concept: Fair CPU sharing based on execution time

1) Uses Completely Fair Scheduler (CFS)
2) Priority from nice value (-20 to +19)
3) Lower nice → higher priority
4) Uses virtual runtime (vruntime)
5) Task with lowest vruntime runs first

Windows Scheduling
Concept: Priority-based preemptive scheduling

1. 32 priority levels (0–31)
    i. 0 → system
    ii. 1–15 → variable
    iii. 16–31 → real-time
2. Highest priority thread runs

Key Behavior
Thread runs until:
→ blocks
→ time slice ends
→ higher priority arrives

Priority Boosting:

1. After waiting
2. Foreground process gets more CPU

Solaris Scheduling

Concept: Priority-based with multiple classes

- ✓ Classes:
  TS, IA, RT, SYS, FSS, FP
- ✓ Each class has own algorithm

Key Points

1. Uses global priority
2. Highest priority thread runs
3. Same priority → Round Robin (RR)

Final Comparison Line

1. Linux: Fair sharing (CFS)
2. Windows: Strong priority + dynamic boost
3. Solaris: Class-based flexible scheduling

Algorithm Evaluation

Select CPU scheduling algorithm by defining criteria (waiting time, turnaround, throughput) and evaluating performance.

1. Deterministic Modeling

Concept: Evaluate using a fixed workload
Same input → compare algorithms
Example: FCFS, SJF, RR
Simple and fast
Limitation: Works only for given data, not general

2. Queueing Models

Concept: Use probability to model process behavior
Arrival & service → random (often exponential)

Calculates:
→ average waiting time
→ queue length
→ CPU utilization

3. Little's Formula

Concept: Give Relation between queue length, arrival rate, waiting time
Formula: $n = \lambda \times W$

n = avg queue length
λ = arrival rate
W = waiting time

Use: Quick performance estimation

Chapter 6: Synchronization

Critical Section Problem

The critical section problem deals with **safe access to shared resources** in a system with multiple processes. Each process may need to execute a segment of code (critical section) that modifies shared data, files, or tables.

The goal is to ensure that **only one process can be in its critical section at a time** to prevent data inconsistency.

A typical process has three parts:

1. **Entry section** – the process asks permission to enter its critical section.
2. **Critical section** – the process executes code that accesses shared resources.
3. **Exit section** – the process leaves the critical section and allows others to enter.
4. **Remainder section** – the process executes code that does not access shared resources.

Requirements for Solving the Critical Section Problem

1. **Mutual Exclusion** – Only one process can be in its critical section at any time.
2. **Progress** – If no process is in the critical section, one of the waiting processes must be allowed to enter without unnecessary delay.
3. **Bounded Waiting** – There must be a limit on how many times other processes can enter their critical sections before a waiting process is granted access. This ensures no process waits indefinitely.

Synchronization Hardware

Synchronization ensures that multiple processes or threads access shared resources safely. Hardware support helps implement **critical sections**, where shared data is accessed without interference.

On **uni-processors**, disabling interrupts can prevent preemption, but this is inefficient on **multiprocessors**, so special hardware instructions are used.

Forms of hardware support:

1. **Hardware instructions** – allow atomic operations like **test-and-set** or **compare-and-swap**.
2. **Atomic variables** – built using hardware instructions to ensure uninterruptible updates to integers, booleans, or other basic types.

Test-and-Set Instruction
Test-and-set is a hardware instruction that atomically reads a boolean variable and sets it to true. It ensures that only one process can acquire a lock at a time.

Example usage:
- A shared boolean variable lock is initialized to false.
- A process repeatedly executes test_and_set(&lock) until it acquires the lock.
- After executing the critical section, it sets lock = false.

Compare-and-Swap Instruction
Compare-and-swap atomically changes a variable only if it matches an expected value. This ensures safe access without race conditions.

Example usage:
- Shared integer lock initialized to 0.
- Process repeatedly executes compare_and_swap(&lock, 0, 1) until it succeeds.
- After the critical section, it resets lock = 0.

Bounded-waiting:
- Processes can track waiting states using arrays and compare-and-swap to ensure **fair access** to critical sections.

Atomic Variables
Atomic variables allow **uninterruptible updates** on shared data. They are often implemented using compare-and-swap.

An atomic variable is called "atomic" because its operations are indivisible—they happen completely or not at all, just like an atom is the smallest indivisible unit. It allows multiple threads to safely access or update the variable at the same time without interference, using hardware instructions like test-and-set or compare-and-swap.

Mutex Locks
Mutex locks are software tools that **simplify critical section protection**. A process acquires the lock before entering the critical section and releases it after execution. They are often implemented using atomic instructions like compare-and-swap. Mutexes may use **busy waiting** and are then called **spinlocks**.

When a mutex uses **busy waiting**, it means that if a thread wants to acquire the lock but it's already held by another thread, it keeps checking repeatedly**(spin)** in a loop until the lock becomes available instead of going to sleep.

Semaphores

Semaphores are integer variables that allow **more sophisticated synchronization** than mutex locks. They provide two atomic operations: **wait()** and **signal()** (originally P() and V()).

wait() – decreases the semaphore. If the value is less than zero, the process waits (or blocks).
signal() – increases the semaphore. If there are waiting processes, one of them is allowed to proceed.

A **semaphore** is like a **traffic signal for processes** that controls who can access a shared resource. The **integer value** of a semaphore shows how many processes can enter or use the resource at a time.

- **wait() operation:** Think of this as a process **asking for permission** to enter. It decreases the semaphore value by 1. If the value becomes negative, it means too many processes are already using the resource, so this process must **wait (block)** until the resource is free.
- **signal() operation:** This is like a process **releasing the resource**. It increases the semaphore value by 1. If there are processes waiting, one of them is **allowed to proceed** into the critical section.

Types of semaphores:

1. **Binary semaphore:** Value can only be 0 or 1. It's like a **single-key lock**, allowing only **one process at a time**—works just like a mutex.
2. **Counting semaphore:** Value can be more than 1. It's like a **limited number of keys** for multiple instances of a resource. For example, if 3 printers are available, the semaphore value is 3. Each process that wants a printer decreases the value; when done, it increases the value.

Example of usage:

```
wait(mutex);
critical_section();
signal(mutex);
```

Semaphore Implementation

- Semaphores can be implemented with **busy waiting**, where a process continuously checks if it can enter, but it wastes CPU cycles if critical sections are long.
- A better approach is **no-busy-wait**, where each semaphore has a waiting queue. Processes that cannot enter are blocked and automatically woken up when the resource becomes free.

Problems with semaphores:

- Using signal before wait can allow multiple processes to enter at the same time, causing conflicts.
- Multiple consecutive waits without a signal can block processes indefinitely (deadlock).
- Omitting wait or signal breaks synchronization.

Semaphores are powerful for synchronization, but must be used carefully to avoid race conditions or deadlocks.

Chapter 8: Deadlocks (From Book Slide)

- *What we will learn here?*

Deadlock Handling

```
│
├─── (A) Deadlock Prevention
│    ├─── Idea: Do not allow deadlock
│    ├─── Break conditions:
│    │    ├─── Mutual Exclusion
│    │    ├─── Hold and Wait
│    │    ├─── No Preemption
│    │    └─── Circular Wait
│    └─── Example:
│         └─── Resource ordering
│
├─── (B) Deadlock Avoidance
│    ├─── Idea: Check before granting request
│    ├─── Goal:
│    │    └─── Keep system in safe state
│    ├─── Algorithms:
│    │    ├─── Single instance → Resource Allocation Graph (RAG)
│    │    └─── Multiple instances → Banker's Algorithm
│    └─── Concepts:
│         ├─── Safe state
│         ├─── Unsafe state
│         └─── Safe sequence
│
└─── (C) Deadlock Detection & Recovery
     ├─── Idea: Allow deadlock, then handle it
     ├─── Detection:
     │    └─── Detect deadlock in system
     └─── Recovery:
          ├─── Terminate processes
          └─── Preempt resources
```

Preemption means a resource can be taken away from a process forcefully or thread without its cooperation.

A semaphore is a synchronization mechanism that controls how multiple threads access shared resources safely.

Deadlock Characterization and Prevention

Mutual Exclusion:

Some resources can only be used by one process at a time. For example, a printer can't be shared; only one process can print at once.

Now think: if a process **has a resource that is non-shareable**, and another process **needs that same resource** to continue, what happens?

1. The second process **must wait** until the first one releases it.
2. If multiple processes are each holding resources and waiting for others (which are also non-shareable), nobody can proceed.

Prevention:

✓ Make resources **sharable** whenever possible.

Example: read-only files can be read by multiple processes at the same time.

✓ Non-shareable resources (like printers) must still use mutual exclusion.

Hold and Wait

This means a process:

1. Is **holding at least one resource**, and
2. Is **waiting to acquire additional resources**.

Why does this cause deadlock?

Imagine this situation:

✓ P1 holds Resource A and waits for Resource B.

✓ P2 holds Resource B and waits for Resource A.

Because they are **holding** resources while **waiting**, neither releases what they already have. So both stay stuck forever.

Prevention:

1. Force processes to **request all resources at once** or **request new resources only when they hold none(all are free)**.

That way, a process never holds one resource while waiting for another.

Drawback: This can cause **low resource utilization** because resources might sit idle waiting for a process to get all it needs.

Another problem: Some processes might **starve**, waiting indefinitely if resources are never enough for their total request.

No Preemption

This means:

1. The system **cannot forcibly take** a resource away from a process.
2. A process must release it voluntarily.

Why does this lead to deadlock?
Imagine:

- ✓ P1 holds Resource A.
- ✓ P2 holds Resource B.
- ✓ P1 needs B.
- ✓ P2 needs A.

If the system were allowed to **take A from P1** and give it to P2 (or vice versa), we could break the cycle.
But because **resources cannot be taken away**, both processes are stuck waiting forever. So without preemption, once a resource is allocated, it stays locked until the process releases it — and that allows deadlock to persist.

Prevention:
If a process requests a resource that's unavailable:

Release all resources it currently holds.
Add the process to the waiting list for the resources it needs.
Restart the process only when all resources (old + new) can be allocated.

This ensures **no process can block others indefinitely**, because the system can forcibly reclaim resources.

Circular Wait

This means there exists a cycle like:

P1 → waiting for resource held by P2
P2 → waiting for resource held by P3
…
…
Pn → waiting for resource held by P1

Why does this cause deadlock?

1. Because it forms a **closed chain of waiting**.
2. Each process is waiting for another in the circle.
3. No one can proceed.
4. No one can release their resource.
5. So the system reaches a permanent waiting state.

Prevention:

- ✓ Impose a **total order on resource types.**
  Example: Assign numbers to resources: R1 < R2 < R3.
- ✓ Processes must request resources **in increasing order**.
- ✓ This prevents a cycle, because no process can wait for a lower-numbered resource while holding a higher-numbered one.

Maximum[i][j] represents the maximum number of instances of resource type j that process i may request during its execution.

Resource-Allocation Graph Scheme(Optional-May Skip)

The **Resource-Allocation Graph (RAG) scheme** is a deadlock-avoidance method used when each resource type has a single instance.

It uses a directed graph containing:

- ✓ **Threads (Ti)** – represented as circles
- ✓ **Resources (Rj)** – represented as rectangles

Types of Edges

1. **Claim Edge (Ti → Rj)**
   - ✓ Dashed line
   - ✓ Indicates that thread Ti may request resource Rj in the future.
   - ✓ Resources must be claimed in advance (a priori).
2. **Request Edge (Ti → Rj)**
   - ✓ Solid line
   - ✓ Shows that Ti is currently requesting Rj.
3. **Assignment Edge (Rj → Ti)**
   - ✓ Indicates that resource Rj is allocated to thread Ti.

When a resource is released, the assignment edge is converted back into a claim edge.

Before allocating a requested resource, the system checks whether granting the request would create a cycle in the graph.

- ✓ If a cycle would form → request is denied.
- ✓ If no cycle forms → request is granted.

Banker's Algorithm

The **Banker's Algorithm** is a **deadlock avoidance** algorithm used when there are **multiple instances of each resource type**.

Key Idea of Banker's Algorithm:

1. Choose any process whose need can be satisfied by available
2. After it finishes, its resources become free
3. These free resources are added with available
4. Continue for the next process with new available

Banker's Algorithm does NOT require one fixed sequence
Any sequence that satisfies Need ≤ Available at each step is valid

<u>Data Structures Used in Banker's Algorithm</u>

Let:

n = number of processes
m = number of resource types

1. Available (Vector of length m)

Available[j] = k

Means: There are **k free instances** of resource type (Rj).

2. Max (n × m matrix)

Max[i][j] = k

Process (Ti) may request **at most k instances** of resource type (Rj).
This is declared before execution starts.

3. Allocation (n × m matrix)

Allocation[i][j] = k

Process (Ti) is currently holding **k instances** of (Rj).

4. Need (n × m matrix)

Need[i][j] = Max[i][j] - Allocation[i][j]

Why It Is Called Banker's Algorithm

Like a banker:

The banker gives loans (resources)
Only if repayment (process completion) is guaranteed
Otherwise, the loan is denied

Safe State (Deadlock Avoidance)

A system is in a **safe state** if it can allocate resources to all threads in some order and still **avoid deadlock**.

Key Idea

1. When a thread requests a resource, the system checks:
2. If granting the request **still allows a safe sequence**, then allocate.
3. Otherwise, **do not allocate immediately** (to prevent deadlock).

Safe Sequence

A **safe sequence** is an ordering of all threads:

<T1, T2, ..., Tn>

Such that for each thread Ti:

1) Its remaining resource needs can be satisfied by:
   a) Currently available resources
   b) resources released by all previous threads (Tj, where j < i)

Safety Algorithm

It checks whether the system is in a **safe state**:

- Can all processes finish without causing deadlock if the current resource allocation is granted?

It uses the **Available**, **Allocation**, and **Need** matrices.

Deadlock Detection

1. Deadlock detection is different from prevention/avoidance:
System allows deadlock to occur.

Periodically runs a **detection algorithm** to identify deadlocked processes.
Uses a **recovery scheme** once deadlock is found (process termination or resource preemption).

2. Single Instance of Each Resource Type

For **resources with only one instance**:
Use Wait-For Graph

1) Make a graph where:
   a) **Nodes** = threads/processes
   b) **Edges** = Ti → Tj if Ti is **waiting for a resource held by Tj**
2) Deadlock occurs if the graph contains a **cycle**.
3) Detection:
   Periodically search for cycles in the wait-for graph.
   If a cycle exists → **deadlock exists**.
4) Time complexity: O(n2), where n = number of threads.

3. Multiple Instances of Each Resource Type

For **resources with multiple instances**, use matrices:

- ✓ **Available (vector length m)**: Number of available resources of each type.
- ✓ **Allocation (n × m matrix)**: Resources currently allocated to each thread.
- ✓ **Request (n × m matrix)**: Additional resources currently requested by each thread.

4. Detection Algorithm (Multiple Instances)

1. **Initialize**:
   - ✓ Work = Available
   - ✓ Finish[i] = false if Allocation[i] ≠ 0, else true
2. **Find a process i** such that:
   - ✓ Finish[i] == false
   - ✓ Request[i] ≤ Work

If no such i exists, go to Step 4.

3. **Simulate completion**:
   - ✓ Work = Work + Allocation[i]
   - ✓ Finish[i] = true
   - ✓ Repeat Step 2
4. **Check Finish vector**:
   - ✓ If any Finish[i] == false → process Ti is deadlocked
   - ✓ If all Finish[i] == true → no deadlock

Complexity: $O(m.n^2)$ where m = resource types, n = processes.

When to Invoke Deadlock Detection

Deadlock detection should be invoked based on how frequently deadlocks are likely to occur and how costly it is to recover from them, running less often if deadlocks are rare and more carefully or frequently if recovery is expensive or deadlocks are common.

How Often to Run

Deadlock detection can be run regularly after some time, or only when a process waits too long, and the choice depends on balancing two things: the extra work (overhead) of running the detection and the problem caused if a deadlock stays for too long.

Challenges if Invoked Arbitrarily

If deadlock detection is run at random times, the system may have multiple deadlocks at once, making it difficult to identify which process caused the problem and making recovery harder because several processes may need to be rolled back.

Recovery from Deadlock

Recovery by Process Termination

Two approaches:

1. **Abort all deadlocked threads:** Simple but costly.
2. **Abort one process at a time:** Continue until the deadlock cycle is broken.

Criteria for choosing which process to abort:

1. Thread priority (high-priority threads are less likely to be aborted)
2. How long the thread has run and how much longer it needs to complete
3. Resources the thread has already used
4. Resources the thread still needs to complete
5. Number of threads that must be terminated if this one is aborted
6. Whether the thread is interactive or batch

Goal: Minimize the overall cost of termination.

Recovery by Resource Preemption

Instead of killing a process:

1. **Select a victim** thread/process – choose one that minimizes cost.
2. **Rollback** its state to a safe point.
3. Starvation

- ✓ A thread may repeatedly be chosen as victim.
- ✓ To prevent this, include the **number of rollbacks** in the cost factor when selecting victims.

Chapter 9: MainMemory(From Book Slide)

1. Program Execution

A program stored on **disk** cannot run directly.
To execute:

1. The program is loaded from disk into **main memory (RAM)**.
2. The OS creates a **process** for it.
3. The CPU executes instructions from memory.

Disk → RAM → CPU → Execution

2. What the CPU Can Directly Access

The CPU can directly access only:

- ✓ **Registers** (fastest, inside CPU, in one cycle)
- ✓ **Main memory (RAM)**

It cannot directly execute instructions from disk.

3. How Memory Looks to the CPU

The memory unit only sees two types of operations:

1. **Read operation**
   - CPU sends an address
   - Memory returns data from that address

2. **Write operation**
   - CPU sends an address
   - CPU sends data
   - Memory stores the data at that address

It does NOT understand programs, variables, loops, etc.
It only understands addresses.

4. Speed Difference Problem

Now comes the important performance issue.

1) **Register access** → 1 CPU clock cycle (very fast)
2) **Main memory access** → many cycles (slow compared to CPU)

Because RAM is slower than the CPU, the CPU must sometimes **wait**.
This waiting is called a **stall**.

5. Why Cache Exists

To reduce this delay, we use:
Cache memory (between CPU and RAM)

1. Why Memory Protection is Required

Multiple processes run at the same time.
Without protection:

- ✓ One process could overwrite another's memory.
- ✓ A program could corrupt the OS.
- ✓ Security vulnerabilities would occur.

So the OS uses: To ensure that a process can only access its own memory.

- ✓ Base and limit registers
- ✓ Paging
- ✓ Segmentation
- ✓ Virtual memory

2. Base and Limit Registers (Simple Hardware Protection)

We use two registers:

Base → starting physical address of the process
Limit → size of the process

Allowed address range:

Base ≤ Physical Address < Base + Limit
Example:

Base = 5000
Limit = 1000

Valid addresses: 5000–5999
Every time a user process tries to access memory:

CPU checks:

Is the address inside the allowed range?
If yes → continue
If no → trap to OS

3. Address Binding (How Addresses Are Assigned)

Address binding is the process of matching program addresses to actual memory locations.

Simple Example:

Think: Program says: "go to address 10" But in memory, it is loaded at 1000

So actual address becomes:

1000 + 10 = 1010

This adjustment = Address Binding

Now the important question from the slide:
Without relocation support, programs must be loaded at address 0000.

Why? Because if the compiler generates absolute addresses, they must match physical memory exactly.

That's inconvenient because:

1. Multiple programs exist
2. Memory is shared
3. Your process cannot always start at 0, other user may already use it.
   So we need flexible binding.

4. Different Forms of Addresses in Computer

A program goes through multiple stages:

Stage 1: Source Code
Addresses are symbolic.
Example: int x;
"x" is just a name.

Stage 2: Compile Time
Compiler produces relocatable addresses.
Example: "14 bytes from beginning of this module"
This means: Offset = 14
Not a real memory location yet.

Stage 3: Linker/Loader
Suppose OS loads program at physical address 74000.
Then: Physical address(Absolute Address) = 74000 + 14 = 74014

The process of converting addresses from one form to another (symbolic → relocatable → physical) is called **address binding**.
Each stage maps one address space to another address space.

5. When Does Binding Happen?

There are three possibilities:
1) Compile-Time Binding

If memory location is known before running:
Compiler generates absolute addresses(Fixed Address).
Example: Program always starts at 1000 (Fixed Place).
Problem: If location changes → must recompile.Very inflexible.

2) Load-Time Binding(Compile Time)

Compiler generates relocatable code.
Loader decides: Where in memory to place the program.
Once loaded, address is fixed.
Cannot move during execution.

3) Execution-Time Binding (Most Important)

Address binding while program is running.
Process can move in memory while executing.
This requires hardware support:

- ✓ Base and limit registers
- ✓ Or modern hardware like paging

Now: Physical Address = Base + Logical Address
Done dynamically during execution.This is how modern systems work.

6. Logical vs Physical Address Space

Logical address (Virtual address)

→ Generated by CPU
→ What the program thinks is the address

Physical address

→ Real location in RAM
→ Seen by memory hardware

But Keep In Mind:

Case 1: Compile-time or Load-time binding
Logical address = Physical address

Case 2: Execution-time binding
Logical address ≠ Physical address

Example:
Logical address = 100
Base register = 5000
Physical address = 5100

Program thinks it is using address 100.
Actually it is using 5100.

7. Address Spaces

Logical address space is the set of all logical addresses generated by a program.

Physical address space is the set of all physical addresses generated by a program.

Memory-Management Unit (MMU)

What it is: A hardware device that **translates logical (virtual) addresses into physical addresses** at runtime.
Why we need it: Programs always use logical addresses (0, 1, 2 ...), but physical memory locations can vary. The MMU handles this translation automatically.

Need to know:
1. **Relocation register:** Contains the starting physical address of the program in memory.
2. **Execution-time binding:** Logical addresses are converted to physical addresses when the CPU references memory.
3. Programs **never see physical addresses** — they just use logical addresses.
4. **Formula:** Address = Logical Address + Relocation Register

Dynamic Loading

- ✓ **Idea:** Not all parts of a program need to be in memory at once.
- ✓ **How it works:** A routine is loaded **only when it is called**.

Benefits:
1. Saves memory space
2. Reduces startup time
3. Useful for rarely used routines

Dynamic Linking

Static linking: All library code is included in the program at load time.

Dynamic linking: Library routines are linked **only when needed at execution time**.

How it works:
1) A small **stub** in memory points to the library routine.
2) When the routine is first called, the OS finds it in memory (or loads it) and updates the stub.

Benefits:
1. Saves memory (shared libraries)
2. Allows updating or patching libraries without recompiling programs

Contiguous Allocation

1. **Idea:** Memory is divided into **two main parts:**
   a. **OS memory** (usually low memory)
   b. **User processes** (high memory)
2. Each process is stored in a **single contiguous block**.
3. **Problem:** Limited flexibility, but simple to implement.

Protection using base and limit registers:

1. **Base register:** Smallest physical address of the process
2. **Limit register:** Maximum logical address the process can access

1. Contiguous Allocation
   1. Process occupies **one continuous block** of memory.
   2. **Pros:** Simple, fast.
   3. **Cons:** Causes **external fragmentation**, inflexible.
   4. Example: Partition

2. Non-Contiguous Allocation
   1. Process can occupy **multiple separate blocks** (like pages).
   2. **Pros:** Flexible, avoids external fragmentation.
   3. **Cons:** Needs **page table**, can have **internal fragmentation**, slightly slower.
   4. Example: Paging, Segmentation

Fixed-Size Partition (Memory divided into equal-sized blocks.

1. **Pros:** Simple, easy to manage.
2. **Cons:** Wastes memory if process is smaller (**internal fragmentation**); large processes may not fit.

First Fit: Take the first hole that fits. Fast, may leave small holes.
Next Fit: Like First Fit, but starts from last allocation.
Best Fit: Take the smallest hole that fits. Minimizes leftover, slower.
Worst Fit: Take the largest hole. Leaves big holes, often wastes memory.

Fragmentation:
Internal Fragmentation

1. Memory **inside a partition** is wasted because the process is smaller than the allocated space.
2. Happens in **fixed or variable partitions**.
3. **Example:** Partition = 100 KB, process = 90 KB → 10 KB wasted.

External Fragmentation

1. Free memory is **scattered in small holes**; total free memory may be enough, but **not contiguous**.
2. Happens in **contiguous allocation**.
3. **Example:** Free blocks = 20 KB + 30 KB + 50 KB, but process needs 60 KB → cannot fit.

Internal Fragmentation Example:(Math)

Page size = 2048 bytes
Process size = 72,766 bytes

Calculation

Full pages = **(Process size/Page size) =** 35 (Taken Integer Value)
Remaining after division = 1086 bytes
Internal fragmentation = 2048 − 1086 = **962 bytes**

Key Points

1) **Worst case internal fragmentation will be** = 1 frame − 1 byte
2) **Average internal fragmentation** ≈ ½ page size

Trade-off

1. **Small pages** → less fragmentation, larger page table
2. **Large pages** → more fragmentation, smaller page table

Variable-Size Partition

1. Memory divided into **partitions sized to each process's needs**.
2. **Pros:** Efficient memory use, avoids internal fragmentation of fixed partitions.
3. **Cons:** Can cause **external fragmentation** (free memory scattered), more complex to manage.

Paging:

1) Paging allows a process to be stored in non-contiguous memory
2) Memory is allocated wherever space is available
3) Eliminates external fragmentation
4) Still has internal fragmentation as Last page may not be fully used
5) Average waste ≈ half page size

Memory Division

Physical memory(RAM) → Frames (fixed size)
Logical memory(Program) → Pages (same size as frames)

Page or frame size is always power of 2

How Program Loading With Paging:

1. A program with N pages needs N free frames (Page = Frame)
2. Pages are loaded into any available frames
3. OS keeps track of free frames

Page Table: Used to map the relation:
Page number → Frame number
It Converts logical address to physical address

Address Translation
Logical address is divided into:
Page number (p) → used to find frame
Offset (d) → position inside page

Physical address = Frame number + Offset

Page Table Implementation: Stored in main memory
1) Page-table base register (PTBR) points to the page table
2) Page-table length register (PTLR) indicates size of the page table

But A New Problem Is:
Each data or instruction access needs 2 memory accesses because:
One for the page table and one for the data / instruction
Solution is → TLB

Translation Lookaside Buffer (TLB) it **is a small, fast hardware cache** used to speed up virtual-to-physical address translation that stores recent page table entries.

1. **Purpose:** Speeds up **logical → physical address translation** in paging.
2. **How it works:**
   a. CPU generates a logical address → check TLB first.
   b. If **hit** → physical address found immediately.
   c. If **miss** → consult page table in memory, then update TLB.
3. **Benefit:** Reduces memory access time, improves performance.

Replacements of TLB entries can be:
1) **LRU:** Least Recently Used Entry will be replaced.
2) **Round-Robin**: Entries will be replaced after a fixed time.
3) **Random:** Randomly Entries will be replaced.

Page Fault: Happens when the page needed by the process is not in main memory. OS must load the page from disk (backing store) into memory. After loading, update the page table and TLB, then resume execution.

Effective Access Time (EAT): Measures average memory access time
If hit ratio is high → faster system
Example:
80% hit → slower (more misses)
99% hit → very fast

Shared Pages:
Shared pages allow multiple processes to use same memory space efficiently.

1) Shared Code

1. A single copy of read-only (re-entrant) code is shared among multiple processes
2. Common in programs like editors, compilers, and window systems
3. Similar to threads sharing the same process memory
4. Helps in saving memory

2) Private Code and Data

1. Each process maintains its own copy of code and data
2. Not shared with other processes
3. Can be placed anywhere in logical memory

Structure of Page Table
Problem: Page tables can become very large for modern systems
Example:

As example a 32-bit address space with 4 KB page size Requires about 1 million entries, Each process may need several MB for page table

Solutions: To reduce size and improve efficiency, different structures are used:

1) Hierarchical Page Tables:

1. Page table is divided into multiple levels
   a) Example: Two-level page table
2. The page table itself is paged
3. Reduces memory usage

2) Hashed Page Tables

1) Uses a hash function to locate page entries
2) Suitable for large address spaces

3) Inverted Page Tables

1. One entry per frame instead of per page
2. Significantly reduces table size
3. Stores mapping using frame → page

Two-Level Paging
Two-level paging is used to reduce the size of large page tables by dividing them into smaller parts.
Instead of one large page table, it uses:

Outer Page Table
Inner Page Table

Example (32-bit system)

Logical address = 32 bits
Page size = 4 KB (2^{12} bytes)

So:

Page number = 20 bits
Offset = 12 bits

Two-Level Division

The 20-bit page number is divided into:

p1 (10 bits) → index into outer page table
p2 (10 bits) → index into inner page table

Address Format

| p1 (10 bits) | p2 (10 bits) | d (12 bits) |

Address Translation Steps

1. Use p1 to access the outer page table
2. Get the address of the inner page table
3. Use p2 to find the frame number
4. Combine frame number with offset (d) to get physical address

64-bit Logical Address Space

Problem

1) Address space becomes very large
2) Two-level paging is not sufficient

Solution

1. Use multi-level paging:
2. Three-level paging
3. Four-level paging

Drawback

1) More levels increase memory access time
2) May require multiple memory accesses for one data access

Hashed Page Tables

Hashed page tables are used for large address spaces (greater than 32-bit) to reduce page table size and improve efficiency.

Working

- ✓ The virtual page number is passed through a hash function
- ✓ It maps to an index in the hash table
- ✓ Each index contains a linked list (chain) of entries

Each Entry Contains

1. Virtual page number
2. Corresponding physical frame number
3. Pointer to next entry (for collision handling)

Inverted Page Table

Instead of one page table per process, use one global table for entire system
One entry for each physical frame

Each Entry Contains

Virtual page number
Process ID (owner of page)

Advantages

Greatly reduces memory usage
Suitable for large address spaces

Disadvantages

Searching is slower
Requires hashing to speed up search
Difficult to implement shared memory

Optimization

Use hash table to reduce search time
Use TLB for faster access

Example: Oracle SPARC Solaris

Features

1. Uses advanced hashing technique
2. Maintains two hash tables:
   a) One for kernel
   b) One for user processes

Address Translation

1. CPU checks TLB
2. If miss → search Translation Storage Buffer (TSB)
3. If found → load into TLB
4. If not found → kernel searches hash table
5. Kernel updates TSB and TLB

Swapping

Swapping is a memory management technique in which a process is temporarily moved from main memory (RAM) to disk (backing store) and later brought back for execution.

Basic Idea
Allows more processes to run than physical memory can hold
Improves CPU utilization
Process execution is paused while swapped out

Backing Store
A fast disk used to store processes temporarily
Must be large enough to hold memory images of processes
Supports direct access to data

Roll Out / Roll In
Used in priority scheduling
Lower-priority process → swapped out
Higher-priority process → loaded into memory

Swap Time Main cost is data transfer time that Depends on:
- Size of process
- Disk speed

Larger process → more time

Ready Queue Contains processes that are Ready to run and Stored in memory or on disk

Important Issues
1) Memory Location
Process may or may not return to same memory location
Depends on address binding method

2) Pending I/O Problem
Cannot swap out process doing I/O
May cause wrong data transfer
Solution: Use kernel buffer (double buffering)

3) Context Switch Time
If swapping is needed → very slow
Example:
Large process → seconds of delay

4) Modern Systems
Full swapping is rarely used
Used only when memory is very low
Common in UNIX, Linux, Windows (in modified form)

Swapping in Mobile Systems

Usually not used because: Flash memory is slow and Limited write cycles
Instead: iOS → asks apps to free memory
Android → kills apps and saves state

Intel 32-bit and 64-bit Architectures

- ✓ Intel processors are widely used
- ✓ IA-32 → 32-bit (Pentium)
- ✓ x86-64 / IA-64 → 64-bit (modern CPUs)

IA-32 Architecture (32-bit)

Memory Management
Supports: Segmentation and Segmentation with Paging

Segmentation
Memory divided into segments
Each segment size: up to 4 GB
Each process: up to 16K segments

Segment Tables LDT (Local Descriptor Table)

1. Private to each process
2. Up to 8K segments

GDT (Global Descriptor Table)

- ✓ Shared among all processes
- ✓ Up to 8K segments

Address Translation (IA-32)
Steps:

1. CPU generates logical address
2. Segmentation Unit
   a) Uses selector (LDT/GDT)
   b) Produces linear address
3. Paging Unit
4. Converts linear → physical address

Flow
Logical Address → Segmentation → Linear Address → Paging → Physical Address

Paging in IA-32
Page sizes:

- ✓ 4 KB
- ✓ 4 MB

Paging unit works like MMU

Page Address Extension (PAE)
Purpose Overcome 4 GB limit of 32-bit systems

Features
- ✓ Uses 3-level paging
- ✓ Page entries become 64 bits
- ✓ Supports up to 64 GB physical memory (36-bit)

Intel x86-64 (64-bit)
Features
Very large address space (theoretical > 16 exabytes)
Practically uses 48-bit addressing

Paging
Uses 4-level paging hierarchy
Page sizes: 4 KB or 2 MB or 1 GB

Addressing
Virtual address: ~48 bits
Physical address: up to ~52 bits

ARM Architecture (Mobile Systems)
- ✓ Used in mobile devices (Android, iOS)
- ✓ Energy efficient

Paging
Page sizes: 4 KB, 16 KB
Large pages: 1 MB, 16 MB (called sections)

Paging Levels
One-level paging → for large sections
Two-level paging → for small pages

TLB Structure
Two levels of TLB:
- ✓ Inner TLB (main)
- ✓ Outer TLBs (separate for instruction & data)

Address Translation
Check inner TLB
- ✓ If miss → check outer TLB
- ✓ If still miss → access page table

Chapter 10: Virtual Memory

Virtual Memory

Virtual memory allows a program to run even if it is larger than physical RAM by loading only necessary parts into memory.
It improves memory utilization and lets multiple processes run efficiently.

Demand Paging

Demand paging means pages are loaded into memory only when they are actually needed, instead of loading the whole process at once.

1. Reduces unnecessary disk I/O
2. Saves memory space
3. Speeds up execution
4. Supports more users/processes

Valid–Invalid Bit

Each page table entry has a bit to indicate whether the page is currently in memory or not.

- **Valid (v):** Page is in memory
- **Invalid (i):** Page is not in memory
- If invalid → causes a **page fault**

Page Fault

A page fault occurs when a process tries to access a page that is not currently in memory. The operating system must load that page before continuing execution.

Steps to Handle a Page Fault

When a page fault occurs, the OS follows a sequence to resolve it.

1. Trap to OS (page fault detected)
2. Check if reference is valid or invalid
3. Find a free frame
4. Load page from disk
5. Update page table (set valid bit)
6. Restart the instruction

Steps to Handle Page Fault

1. Process tries to access a page → **page fault occurs**
2. OS checks:
   - Invalid reference → **abort program**
   - Valid but not in memory → continue
3. Find a **free frame**
4. Load page from disk into frame
5. Update page table (**set valid bit = v**)
6. Restart the instruction

Aspects of Demand Paging

Demand paging can be implemented in different ways depending on how aggressively pages are loaded into memory.
In the **extreme case (pure demand paging)**, a process starts with **no pages in memory**.

- The first instruction itself causes a **page fault**
- Every page is loaded only when accessed
- This is called **pure demand paging**

In practice, a single instruction may need multiple pages.

- Example: instruction + data (operands) + result location
- So, one instruction can cause **multiple page faults**

Demand paging requires hardware support to work efficiently.

- Page table with **valid/invalid bit**
- **Secondary storage (swap space)**
- Ability to **restart interrupted instruction**

Free-Frame List

When a page fault occurs, the OS must load the required page into a free memory frame.
To manage this efficiently, the OS keeps a **free-frame list**, which is a pool of available frames.

- When needed, a frame is taken from this list
- At system startup, all frames are initially free

Operating systems often use **zero-fill-on-demand**.

- Before allocating a frame, it is filled with zeros
- Ensures no leftover data from previous processes

Page Replacement

Page replacement is used when memory is full and a new page needs to be loaded, so the OS must remove an existing page.

- If a page is modified (**dirty bit = 1**), it must be written back to disk
- If not modified, it can be removed directly
- Enables large virtual memory even with small physical memory

Need for Page Replacement

When all memory frames are occupied, the OS must replace an existing page to load a new one. Without page replacement, demand paging cannot continue once memory is full.

Basic Page Replacement Steps

When a page fault occurs and no free frame is available, the OS follows a process:

1. Locate required page on disk
2. Check for free frame
   a. If available → use it
   b. If not → select a **victim page**
3. If victim is dirty → write it to disk
4. Load new page into free frame
5. Update tables and restart instruction

Page Replacement Algorithms

These algorithms decide **which page to remove** to minimize page faults.
They are evaluated using a **reference string** (sequence of page numbers).

Goal: **reduce page fault rate**
Performance depends on number of frames

FIFO (First-In, First-Out)

FIFO replaces the **oldest loaded page** in memory.

1) Simple using a queue
2) May cause **Belady's Anomaly** (more frames → more faults)
3) Not always efficient

Optimal Algorithm

Optimal replaces the page that will **not be used for the longest time in future**.

1. Gives **minimum page faults**
2. Not practical (future cannot be predicted)
3. Used as a benchmark

LRU (Least Recently Used)

LRU replaces the page that has **not been used for the longest time in the past**.

1) Based on past usage
2) Better than FIFO
3) Widely used

Implementation methods:

1. Counter (track last used time)
2. Stack (move recently used page to top)

LRU Approximation

Since exact LRU is costly, simpler methods are used.

1) Use **reference bit (0 or 1)**
2) If bit = 0 → page can be replaced

Second-Chance Algorithm

This improves FIFO by giving a page a second chance if it was recently used.

1. If reference bit = 1 → reset it and skip
2. If reference bit = 0 → replace it

Also called the **clock algorithm**.

Enhanced Second-Chance

This considers both **reference bit and modify (dirty) bit**.

Pages are grouped as:

1. (0,0) → best to replace
2. (0,1) → needs write to disk
3. (1,0) and (1,1) → recently used, avoid

Counting Algorithms

These use the number of times a page is accessed.

1) **LFU (Least Frequently Used):** remove least used page
2) **MFU (Most Frequently Used):** remove most used page

(Not commonly used)

Page Buffering

The OS keeps some **free frames ready** to improve performance.

1) Load new page quickly into free frame
2) Write modified pages later when system is idle
3) Can reuse pages if needed again

Applications & Page Replacement

Page replacement is based on guessing future usage, but some applications know better.

1. Databases may manage memory themselves
2. **Double buffering** can occur (OS + application both keep copies)
3. **Raw disk access** allows applications to bypass OS buffering

Thrashing

Thrashing occurs when a process spends more time **swapping pages in and out of memory** than actually executing. This happens when the process does not have enough pages (frames) to run efficiently.

- Very **high page-fault rate**
- Constant page loading and replacing
- CPU stays mostly **idle (low utilization)**
- OS may wrongly increase multiprogramming (adds more processes)
- Makes the problem even worse

Why Thrashing Happens

Thrashing occurs when the memory given to a process is **less than what it actually needs**.

- A process needs a set of pages (called **locality**)
- If **locality size > available memory**, pages keep replacing each other
- So the same pages are loaded again and again

Demand Paging & Locality

Demand paging works well because of **locality of reference**.

- Programs use a small group of pages repeatedly
- These groups are called **localities**
- A process moves from one locality to another over time

How to Reduce Thrashing

Thrashing can be controlled by managing memory allocation properly.

- Use **local page replacement** (replace pages within same process)
- Use **priority-based allocation**
- Give enough frames to each process
- Reduce number of active processes if needed

Chapter 13: File-System Interface

File Concept

A file is a **collection of related data stored on disk** and treated as a single unit.

1. Can store **data** (text, numbers, binary) or **programs**
2. Content is defined by the creator
3. Examples: text file, source code, executable

File Attributes

File attributes are **information about a file used by the OS to manage it**.

1) Name (human-readable)
2) Identifier (unique ID that us used by os)
3) Type (text, binary, etc.)
4) Location (disk address)
5) Size
6) Protection (permissions)
7) Time, date, user info (data for protection, security, and usage monitoring)

File Operations

These are the basic actions that can be performed on a file.

1. Create, read, write
2. Delete, truncate
3. Open (load info to memory)
4. Close (save info back to disk)
5. Seek (move position inside file)

Open Files

When a file is opened, the OS keeps track of it using some data structures.

1) **Open-file table** (list of open files)
2) File pointer (current position)
3) **Open count** (how many processes use it)
4) Access rights
5) Disk location info

File Structure

File structure defines **how data is organized inside a file**.

1. **Simple:** sequence of bytes
2. **Record-based:** fixed or variable length
3. **Complex:** formatted documents or executable files

Access Methods

Access methods define **how data is read or written in a file**.

Sequential Access
Data is accessed **one after another in order**.

1. read next, write next
2. simple and common

Direct Access
Data is accessed **at any position directly**.

1. read n, write n
2. faster for random access
3. Allows jumping to any block (standard for HDDs)

Indexed Access
Uses an **index to quickly locate data.**

1) Like searching using a table of contents
2) Improves performance

Disk Structure
Disk is divided into parts to organize data efficiently.

- ✓ Disk → partitions(Subdivided)
- ✓ Each partition = **volume**
- ✓ Can save raw or formatted with file system
- ✓ OS keeps track using directory/volume table

Directory Structure
A directory is a **collection of files (node) and their information**.

- ✓ Stored on disk
- ✓ Helps organize and locate files

Directory Operations

1. Search, create, delete
2. List files
3. Rename
4. Traverse

Directory Organization
Different ways to arrange directories for better management.

Single-Level

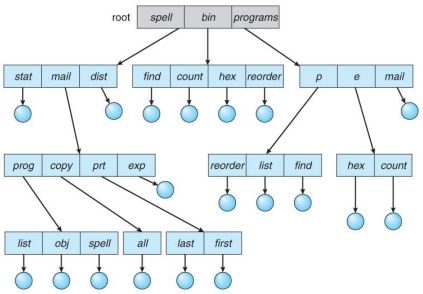
1. One directory for all files and for all users
2. Simple but causes naming problems
3. Have grouping problem

Two-Level

1) Separate directory per user
2) Same file names allowed for different user
3) No grouping capability
4) Support efficient searching

Tree Structure

- ✓ Hierarchical (folders inside folders)
- ✓ Most common and flexible



Graph Structure

- ✓ Allows sharing files between directories
- ✓ May create complexity (cycles)

Current Directory

The current directory is the **working location where operations happen by default**.

1. CMD Commands work relative to this directory
2. Example: create/delete files in current folder

Protection

Protection controls **who can access a file and how**.

1. Read, write, execute
2. Append, delete, list

Access Control in UNIX

Three classes of users: Owner, group, public
Permissions(Mode of Access): read (r), write (w), execute (x)
Example: rwx = 111 = 7

Memory-Mapped Files

Memory-mapped files allow a file to be accessed **as if it is part of main memory**.

1) Faster file access
2) No need for explicit read/write operations